The Socket API

The socket API is a mechanism that providesa low level of abstraction for IPC. Although application programmers seldom have to code at this *level,* the understandingof socket API is important for at least two reasons. First, the upper-layer facilities are built on top of the socket API; that is, they are implemented using theoperations provided in the socket API. Second, for applications that place a premiumon response time or that run on a platform with limited resources, thesocket API may be the most appropriate, or may *even* be the only available, facilityfor IPC.

4.1 Background

The socket API first appeared in the early 1980s as a program library in a versionof the UNIX operating system known as Berkeley Unix (BSD 4.2) to provide thefunctionalities for IPC. Today socket APIs are supported on all major operatingsystems. On UNIX-based systems such as BSD or Linux, the API is part of thekernel, or core, of the: operating system. On personal computer operating systemssuch as MS-DOS,. Windows NT (and its variants), Mac-OS, and OS/2, theAPIis provided as program libraries. (O~ Windows systems, the API is known asWinsock.) Java, a language designed with network programming in mind, provides the socket API as part of the language's core classes. These APIs all sharethe same message-passing model and very similar syntax.

Socket is a term borrowed from telephone communications. In the early days oftelephony (prior to the twentieth century), a person wishing to make a call toanother person had to go through a human operator, who manually establisheda connection by physically inserting the two ends of a cable into two specificreceptacles, each assigned to one of the two parties, on a panel of sockets.

Disconnection also had to be carried out by the operator manually. Thismetaphor was the basis of the socket API for interprocess communications.

4.2 The Socket Metaphor in IPC

Borrowing from the terminology of telephony, the designer of the socket APIhas provided a programming construct termed a *socket.* A process wishing tocommunicate with another process must create an instance of such a construct. Unlike early telephony, however, the communication betweenthe parties may be connection-oriented or connectionless. For clarity, we willpresent the connectionless socket API first.

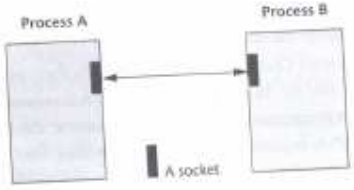


Figure 4.1 The conceptual model of the socket API.

4.3 The Datagram Socket API

There are two keyprotocols at the transport layer of the Internet architecture: User Datagram protocol (UDP) and Transmission Control Protocol (TCP).

The User Datagram Protocol (UDP) allows a packet to be transported (that is,to be sent or received at the transport layer) using connectionless communication.The data packet thus transported is called a datagram. In accordance with connectionless communication, each datagram transported is individually addressed and routed, and may arrive at the receiver in any order. For example,if processl on host A sends messages transported in datagrams ml, *m2*successivelyto process 2 running on host B, the datagrams may be transported overthe network via different routes, and they may arrive at the receiving process in either of the two orders: m1-m2 or m2-m1.

The transmission control Protocol (TCP) is connection oriented and transportsa stream of data over a logical connection established between the senderand the receiver. By virtue of the connection, data sent from a sender to areceiver can be guaranteed to be received in the same order that they were sent.

For example, if process 1 on host A sends messages transported in *m1, m2,* successivelyto process 2 running on host B, the receiving process may assume thatthe messages will be delivered to it in the order *m1-m2*and not *m2-m1.*

The Java socket API, as with all other socket APIs, provides socket programmingconstructs that make use of either the UDP or TCP protocol. Sockets that useUDPfor transport are known as datagram sockets, while sockets that use TCPare termed stream sockets.

The Connection less Datagram Socket

It may surprise you, but datagram sockets can support both connectionless andconnection-oriented communication at the application layer. This is so because even though datagrams are sent or received without thenotion of connections at the transport layer, the run-time support of the socketAPI can create and maintain logical connections for datagrams exchangedbetween two processes.

In Java, two classes are provided for the datagram socket API:

1. The *DatagramSocket*class for the sockets
2. The *DatagramPacket*class for the datagrams exchanged

A process wishing to send or receive data using this API must instantiate a*DatagramSocket*object, or a socket in short. Each socket is said to be bound to aUDPport of the machine that is local to the process (that is, the machine onwhich the process is executing). In IPv4 the validport numbers are 0 through 65,535, with 0 through 1023 reserved for wellknownservices.

To send a datagram to another process,a process must create an object that represents the datagram itself. This object can be created by instantiating a *DatagramPacket*object that carries (1) a reference to a byte array that contains the payload data, and (2) the destination address (the host ID and port number to which the receiver's socket is bound, Once the*DatagramPacket*object is created and loaded with the payload data and destination, the sending process then invokes a call to the *send* method in the*DatagramSocket*object, specifying a reference to the *DatagramPacket*object as an argument.

At the receiving process, a *DatagramSocket*object must also be instantiated- andund to a local port; the port number must agree with that specified in thedatagram packet of the sender. To receive datagrams sent to the socket, theprocess creates a *DatagramPacket*object that

references a byte array and calls a*receive* method in its *Datagramsocket*object, specifying as argument a referenceto the *DatagramPacket*object.

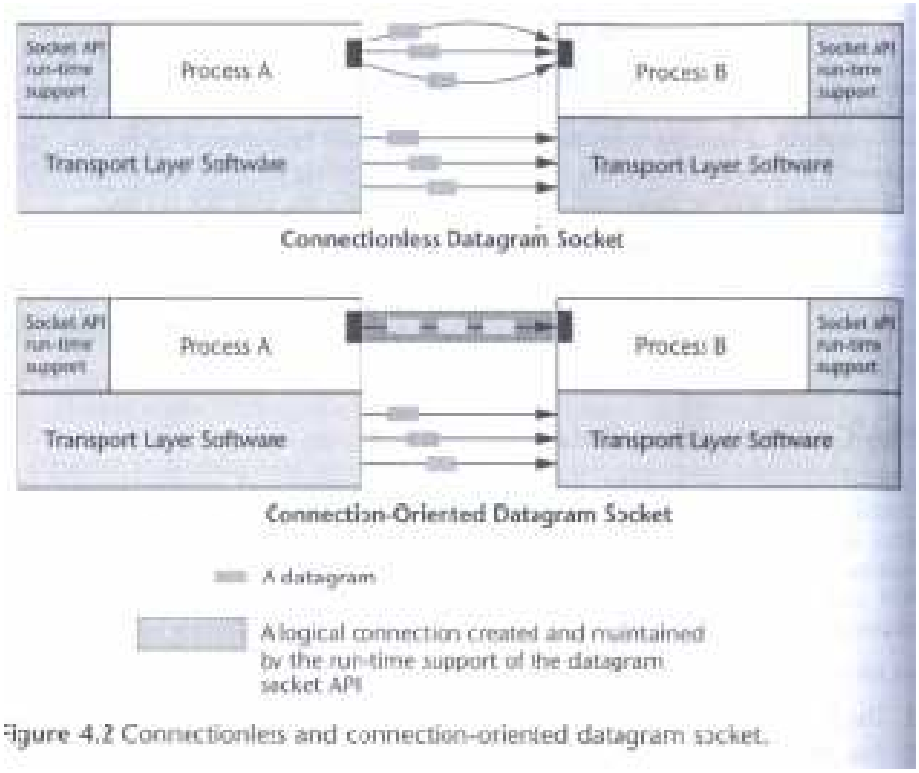


Figure 4.2 Connectionless and connection-oriented datagram socket.

Figure 4.3 illustrates the data structures in the programs for the two processes,while Figure 4.4 illustrates the program flow in the two processes.

With connectionless sockets, a socket bound to a process can be used to senddatagrams to different destinations. It is also possible for multiple processes tosimultaneously send datagrams to the same socket bound to a receiving process,in which case the order of the arrival of these messages will be unpredictable, in accordance with the underlying UDP protocol.

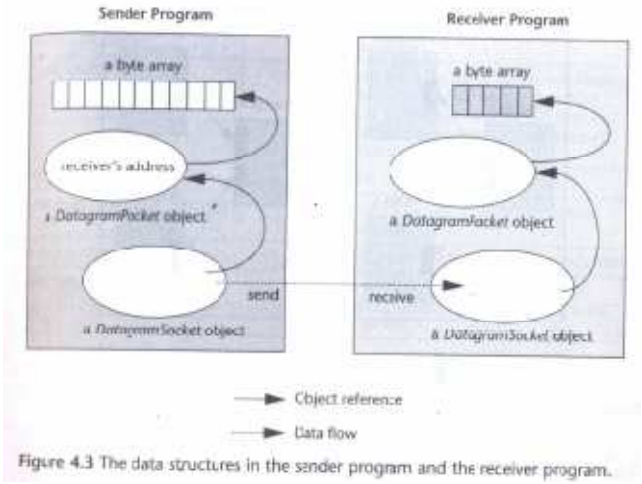


Figure 4.3 The data structures in the sender program and the receiver program.

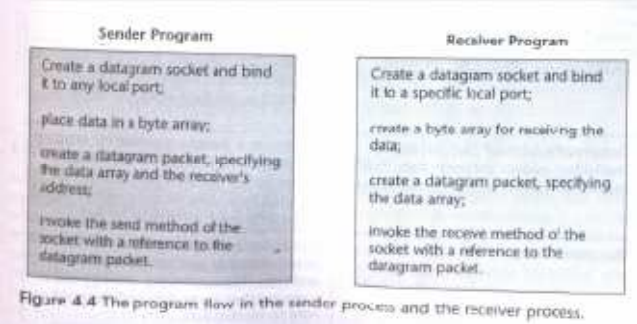


Figure 4.4 The program flow in the sender process and the receiver process.

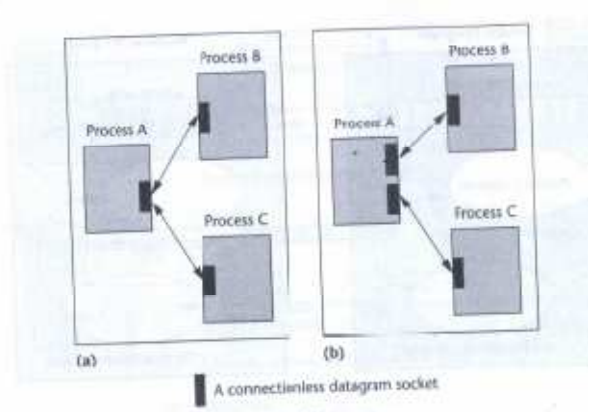


Figure 4.5 Connectionless datagram sockets.

4.4 The Stream-Mode Socket API

Whereas the datagram socket API supports the exchange of discrete units ofdata (that is, datagrams), the stream-mode socket API provides a model of datatransfer based on the stream-mode I/O of the Unix operating systems. By definition,a stream-mode socket supports connection-oriented communicationonly.

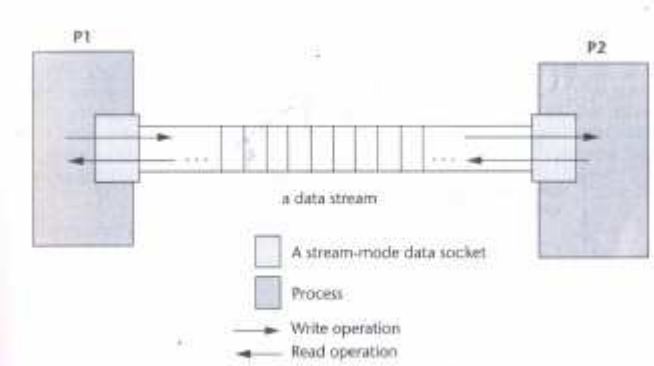


Figure 4.15 Using a stream-mode socket for data transfer.

In stream-mode input-output, data is transferred using the concept of a continuous data stream flowing from a source to a destination (also called a *sink).* Datais inserted, or written, into a stream by a process that controls the source, anddata is extracted, or read, from the stream by a process attached to the destination.

Note that thecontinuous nature of a stream allows data to be inserted and extracted from thestream at different rates.The stream-mode socket API is an extension of the stream-modeI/O model. Using the API, each of two processes individually creates a streammodesocket. A connection between the sockets is then formed. Data, as astream of characters, are written into the sender's socket, which can then be read by the receiver via its socket. This is similar to the connection-orienteddatagram socket API that we have already studied, except for the difference inthe discrete nature of the data transported in datagram sockets.

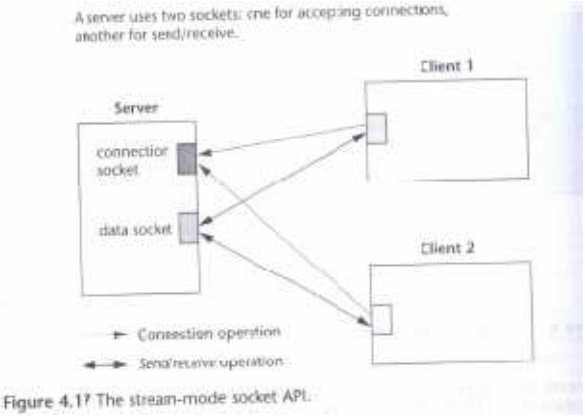


Figure 4.17 The stream-mode socket API.

In Java, the stream-mode socket API is provided with two classes: *ServerSocket*and *Socket.* The term *Server* comes from the client-server paradigm, for which theAPI was designed. The syntax of the API isintertwined with the client-server paradigm. For now we will study the APIindependent of that paradigm.

With the stream-mode socket API, there are two types of sockets:

• The first type of sockets, provided via the *ServerSocket*class, is for accepting connections.(Connection Sockets)

• The other type of sockets, provided via the *Socket* class, is for data exchange. (data sockets)

Using this API, a process known as the server establishes a connection socketand then listens for connection requests from other processes. Connectionrequests are accepted one at a time. When a connection is accepted, a datasocket is created for the connection. Through the data socket, the server processcan read from and/or write to the data stream. When the communication sessionbetween the two processes is over, the data socket is closed, and the serveris free to accept the next connection request via the connection socket.

A process that wishes to communicate with the server is known as a client. Aclient creates a socket; then, via the server's connection socket, it makes arequest to the server for a connection. Once the request is accepted, the client'ssocket is connected to the server's data socket so that the client may proceed toread from and/or write to the data stream. When the communication sessionbetween the two processes is completed, the data socket is closed.

Operations and Event Synchronization

There are two main classes in the stream socket API: the *ServerSocket*class andthe *Socket* class. The *ServerSocket*class is for the establishment of connections,while the *Socket* class is for the transfer of data.

For event synchronization, the following operations are blocking:

• Accept (accepting a connection). If no request is waiting, the server processwill be suspended until a request for connection arrives .

• Reading from the input stream associated with a data socket. If the amountof data requested is not currently present in the data stream, the readingprocess will be blocked until a sufficient amount of data has been writteninto the data stream.

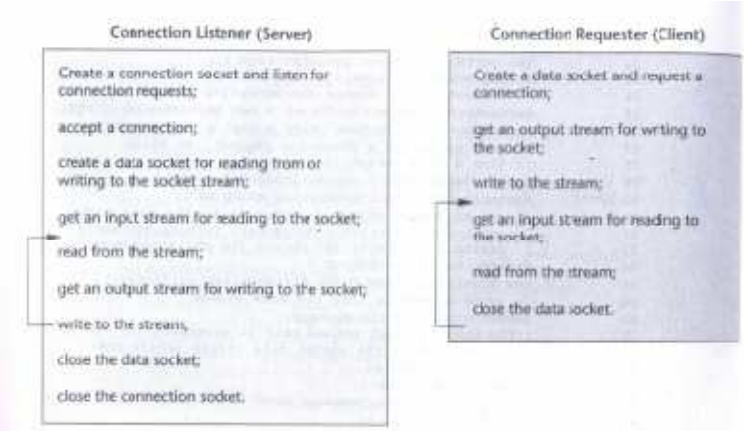


Figure 4.18 Program flow n a connection listener and a connection requestor.

4.5 Sockets with Nonblocking I/O Operations

As mentioned, the APIs introduced in this chapter are basic socket APIs, whichprovide asynchronous (nonblocking) *send* operations and synchronous (blocking)*receive* operations. Using these APIs, a process that reads from a socket issubject to blocking. To maximize concurrency, threads can be used so that aread operation is performed in a waiting thread while a separate thread remainsactive for processing other tasks.

However, in some applications that requireextensive use of threads, the overhead incurred can be detrimental to the performanceor, worse yet, the viability, of the application. Alternatively, there aresocket APIs that provide nonblocking I/O operations. Using such an API, neither*send* nor *receive* will result in blocking, it is necessaryfor the receiving process to use a listener to be notified of the arrival ofthe data. Asynchronous sockets are available with Winsock, and, as of version1.4, Java also provides a new I/O package, java.nio (NIO), that offers socketswith nonblocking I/O operations. The syntax for the new package is considerablymore complex than that for the basic API.

4.6 Secure Socket API

Secure socket APIs exist, which are socket APIs enhanced with data securitymeasures.Using conventional socket APIs, data is transmitted as bit streams over networklinks. The bit streams, if intercepted by means of tools such as network protocolanalyzers, can be decoded by someone who has knowledge of the data rep representation of the data exchanged. Hence it is risky to use sockets to transmitsensitive data, such as credit information and

authentication data. To addressthe problem, protocols have been introduced to secure the data transmittedUsing socket APIs.

The Secure Socket Layer

Secure Sockets Layer (SSL) wasa protocol developedby the Netscape Communications Corporation for transmitting private documents over the Internet. An SSL API has methods or functionssimilar to the socket API, except that data is encrypted before it is transmittedover an SSL connection. SSL IS supported by modern browsers. When runwith the SSL protocol selected, these browsers will transmit encrypted data confidential user information, such as credit card numbers. By convention, the URL of a Web page that requires an SSL connection starts with https: instead http:

The Java Secure Socket Extension

The Java" Secure Socket Extension (J*SSE)* is a set of Java packages that enable secure Internet communications. It implements a version of SSL and TLS (Transport Layer Security) protocols and includes functionalities of data encryption, server authentication, message integrity, and optional client authentication.